In [1]:
```
pip install pandas
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in C:\Users\HP\AppData\Roaming\Python\Pytho
n314\site-packages (3.0.5)
Requirement already satisfied: numpy>=2.3.3 in C:\Users\HP\AppData\Roaming\Python
\Python314\site-packages (from pandas) (2.5.2)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\HP\AppData\Roam
ing\Python\Python314\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\HP\AppData\Roaming\Python\Pytho
n314\site-packages (from pandas) (2026.3)
Requirement already satisfied: six>=1.5 in C:\Users\HP\AppData\Roaming\Python\Pyt
hon314\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import pandas as pd
df = pd.read_csv(
"training_set_rel3.tsv",
sep="\t",
encoding="latin1"
)
print(df.shape)
print(df.columns)
```

```
(12976, 28)
Index(['essay_id', 'essay_set', 'essay', 'rater1_domain1', 'rater2_domain1',
       'rater3_domain1', 'domain1_score', 'rater1_domain2', 'rater2_domain2',
       'domain2_score', 'rater1_trait1', 'rater1_trait2', 'rater1_trait3',
       'rater1_trait4', 'rater1_trait5', 'rater1_trait6', 'rater2_trait1',
       'rater2_trait2', 'rater2_trait3', 'rater2_trait4', 'rater2_trait5',
       'rater2_trait6', 'rater3_trait1', 'rater3_trait2', 'rater3_trait3',
       'rater3_trait4', 'rater3_trait5', 'rater3_trait6'],
      dtype='str')
```

In [3]:
```
print(df[['essay_set', 'essay', 'domain1_score']].head(3).to_string())
```

	essay_set	essay	domain1_score
0	1	Dear local newspaper, I think effects computers have on people are great learning skills/affects because they give us time to chat with friends/new people, helps us learn about the globe(astronomy) and keeps us out of troble! Thing about! Dont you think so? How would you feel if your teenager is always on the phone with friends! Do you ever time to chat with your friends or buisness partner about things. Well now - there's a new way to chat the computer, theirs plenty of sites on the internet to do so: @ORGANIZATION1, @ORGANIZATION2, @CAPS1, facebook, myspace ect. Just think now while your setting up meeting with your boss on the computer, your teenager is having fun on the phone not rushing to get off cause you want to use it. How did you learn about other countrys/states outside of yours? Well I have by computer/internet, it's a new way to learn about what going on in our time! You might think your child spends a lot of time on the computer, but ask them so question about the economy, sea floor spreading or even about the @DATE1's you'll be surprise at how much he/she knows. Believe it or not the computer is much interesting then in class all day reading out of books. If your child is home on your computer or at a local library, it's better than being out with friends being fresh, or being perpressured to doing something they know isnt right. You might not know where your child is, @CAPS2 forbidde in a hospital bed because of a drive-by. Rather than your child on the computer learning, chatting or just playing games, safe and sound in your home or community place. Now I hope you have reached a point to understand and agree with me, because computers can have great effects on you or child because it gives us time to chat with friends/new people, helps us learn about the globe and believe or not keeps us out of troble. Thank you for listening.	8
1	1	Dear @CAPS1 @CAPS2, I believe that using computers will benefit us in many ways like talking and becoming friends will others through websites like facebook and mysace. Using computers can help us find coordibates, locations, and able ourselfs to millions of information. Also computers will benefit us by helping with jobs as in planning a house plan and typing a @NUM1 page report for one of our jobs in less than writing it. Now lets go into the wonder world of technology. Using a computer will help us in life by talking or making friends on line. Many people have myspace, facebooks, aim, these all benefit us by having conversations with one another. Many people believe computers are bad but how can you make friends if you can never talk to them? I am very fortunate for having a computer that can help with not only school work but my social life and how I make friends. Computers help us with finding our locations, coordibates and millions of information online. If we didn't go on the internet a lot we wouldn't know how to go onto websites that @MONTH1 help us with locations and coordinates like @LOCATION1. Would you rather use a computer or be in @LOCATION3. When your supposed to be vacationing in @LOCATION2. Million of information is found on the internet. You can as almost every question and a computer will have it. Would you rather easily draw up a house plan on the computers or take @NUM1 hours doing one by hand with ugly erazer marks all over it, you are garrenteed that to find a job with a drawing like that. Also when appling for a job many workers must write very long papers like a @NUM3 word essay on why this job fits you the most, and many people I know don't like writing @NUM3 words non-stopp for hours when it could take them I hav an a computer. That is why computers we needed a lot now adays. I hope this essay has impacted your descion on computers because they are great machines to work with. The other day I showed my mom how to use a computer and she said it was the greatest invention sense sliced bread! Now go out and buy a computer to help you chat online with friends, find locations and millions of information on one click of the button and help your self with getting a job with neat, prepared, printed work that your boss will love.	9
2	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more people use computers, but not everyone agrees that this benefits society. Those who support advances in technology believe that computers have a positive effect on people. Others have different ideas. A	

```
great amount in the world today are using computers, some for work and spme for t
he fun of it. Computers is one of mans greatest accomplishments. Computers are he
lpful in so many ways, @CAPS4, news, and live streams. Don't get me wrong way to
much people spend time on the computer and they should be out interacting with ot
hers but who are we to tell them what to do. When I grow up I want to be a author
or a journalist and I know for a fact that both of those jobs involve lots of tim
e on time on the computer, one @MONTH1 spend more time then the other but you kno
w exactly what @CAPS5 getting at. So what if some expert think people are spendin
g to much time on the computer and not exercising, enjoying natures and interacti
ng with family and friends. For all the expert knows that its how must people mak
e a living and we don't know why people choose to use the computer for a great am
ount of time and to be honest it's non of my concern and it shouldn't be the so c
alled experts concern. People interact a thousand times a day on the computers. C
omputers keep lots of kids of the streets instead of being out and causing troubl
e. Computers helps the @ORGANIZATION1 locate most wanted criminals. As you can se
e computers are more useful to society then you think, computers benefit society.
7
```

In [4]:
```
print(df['essay_set'].value_counts().sort_index())
```

```
essay_set
1    1783
2    1800
3    1726
4    1770
5    1805
6    1800
7    1569
8     723
Name: count, dtype: int64
```

In [5]:
```
print(df[['essay_set', 'domain1_score']].groupby('essay_set').agg(['min', 'max',
```

```
          domain1_score
                    min max       mean
essay_set
1                     2  12   8.528323
2                     1   6   3.415556
3                     0   3   1.848204
4                     0   3   1.432203
5                     0   4   2.408864
6                     0   4   2.720000
7                     2  24  16.062460
8                    10  60  36.950207
```

In [6]:
```
for essay_set in sorted(df['essay_set'].unique()):
    print("Essay Set:", essay_set)
    print(df[df['essay_set'] == essay_set]['essay'].iloc[0][:500])
    print("-" * 80)
```

Essay Set: 1
Dear local newspaper, I think effects computers have on people are great learning skills/affects because they give us time to chat with friends/new people, helps us learn about the globe(astronomy) and keeps us out of troble! Thing about! Dont you think so? How would you feel if your teenager is always on the phone with friends! Do you ever time to chat with your friends or buisness partner about things. Well now - there's a new way to chat the computer, theirs plenty of sites on the internet to
---
Essay Set: 2
Certain materials being removed from libraries such as books, music and magazines, shouldn't be removed from the libraries. It gives people a chance to understand how the real world @CAPS2. Having certain materials such as books and music definitly should not be removed, because most books and music can show most people how bad the statement in the book @CAPS2 or how bad the lyrics are in a song, and help that person to avoid that type of thing that the book or song @CAPS2 saying to the read
---
Essay Set: 3
The features of the setting affect the cyclist in many ways. The features of the setting that affect the cyclist is the lact of information on were to go and the lack of water. This was a problem because he needed water for his trip and directions on were to go.

Essay Set: 4
The author concludes the story with this because gardens cannot grow with snow on the ground. So she she figurers when all the snow is melted she□ll try to grow another one. I feel as if she wasn□t happy enough to grow another garden I think she should be more interested in growin it then disappointed.

Essay Set: 5
In this memoir of Narciso Rodriguez, @PERSON3's life, the mood containing it all, was greatful, and showed how his parents gave him love through his whole life. Both his parents were born and raised in Cuba, and in 1956, they both moved to the United States. Starting their lives over again, and taking any job they could find. Then in 1961, @PERSON2, @CAPS1. was born. Both Parents raised him with love, and care, and introducing his Cuban background into his life. As @PERSON2, was telling h
---
Essay Set: 6
There were many obstacles that the builders faced in attempting to have dirigibles dock at the Empire State Building. Safety was an issue with the dirigibles because of the highly flammable gas hydrogen. So, if the dirigibles explode over New York many people could be killed in the very populated. Another problem is keeping the derigible @CAPS1 so people can get off while the wind moves the back end of itself. The only way was to tie lead weights down in the back of the dirigible, but it would b
---
Essay Set: 7
Patience is when your waiting .I was patience when in line waiting for lunch .I didn□t c ut any one to eat .I was standing and waiting for my turn .Patience ,some people don□t have it .Lots of people just cut or yell at you because they don□t have any patience. Sometimes people will push you out of their way .They only do that because they don□t have patience at all. Patience is what people need .People need patience because lots o f feelings get hurt .Everyone should have patience.

Essay Set: 8
 A long time ago when I was in third grade I had a friend @PERSON2 who's mom was in a bad mood. She never laughed and she never smiled. Every time I saw her I wou

```
ld smile at her and all she would do was frown and keep walking. At first I did
n't know she was a grouch i just thought she didn't like me or something.When @PE
RSON2 told me his mom was a grouch I started to laugh and laugh. He asked me what
was so funny i told him that i thought his mom didn't like me or something becaus
e every time I
--------------------------------------------------------------------------------
```

In [7]:
```
print("Missing essays:", df['essay'].isnull().sum())
```

```
Missing essays: 0
```

In [8]:
```
print("Duplicate essays:", df['essay'].duplicated().sum())
```

```
Duplicate essays: 4
```

In [9]:
```
import re

def clean_essay(text):
    # Remove anonymization tags such as @CAPS1, @PERSON2, @DATE1
    text = re.sub(r'@\w+\d*', '', text)

    # Replace multiple spaces/newlines with a single space
    text = re.sub(r'\s+', ' ', text)

    # Remove leading/trailing spaces
    text = text.strip()

    return text

df['clean_essay'] = df['essay'].apply(clean_essay)

print(df[['essay', 'clean_essay']].head(3).to_string())
```

essay
clean_essay
0
Dear local newspaper, I think effects computers have on people are great learning skills/affects because they give us time to chat with friends/new people, helps us learn about the globe(astronomy) and keeps us out of troble! Thing about! Dont you think so? How would you feel if your teenager is always on the phone with friends! Do you ever time to chat with your friends or buisness partner about things. Well now - there's a new way to chat the computer, theirs plenty of sites on the internet to do so: @ORGANIZATION1, @ORGANIZATION2, @CAPS1, facebook, myspace ect. Just think now while your setting up meeting with your boss on the computer, your teenager is having fun on the phone not rushing to get off cause you want to use it. How did you learn about other countrys/states outside of yours? Well I have by computer/internet, it's a new way to learn about what going on in our time! You might think your child spends a lot of time on the computer, but ask them so question about the economy, sea floor spreading or even about the @DATE1's you'll be surprise at how much he/she knows. Believe it or not the computer is much interesting then in class all day reading out of books. If your child is home on your computer or at a local library, it's better than being out with friends being fresh, or being perpressured to doing something they know isnt right. You might not know where your child is, @CAPS2 forbidde in a hospital bed because of a drive-by. Rather than your child on the computer learning, chatting or just playing games, safe and sound in your home or community place. Now I hope you have reached a point to understand and agree with me, because computers can have great effects on you or child because it gives us time to chat with friends/new people, helps us learn about the globe and believe or not keeps us out of troble. Thank you for listening.
Dear local newspaper, I think effects computers have on people are great learning skills/affects because they give us time to chat with friends/new people, helps us learn about the globe(astronomy) and keeps us out of troble! Thing about! Dont you think so? How would you feel if your teenager is always on the phone with friends! Do you ever time to chat with your friends or buisness partner about things. Well now - there's a new way to chat the computer, theirs plenty of sites on the internet to do so: , , , facebook, myspace ect. Just think now while your setting up meeting with your boss on the computer, your teenager is having fun on the phone not rushing to get off cause you want to use it. How did you learn about other countrys/states outside of yours? Well I have by computer/internet, it's a new way to learn about what going on in our time! You might think your child spends a lot of time on the computer, but ask them so question about the economy, sea floor spreading or even about the 's you'll be surprise at how much he/she knows. Believe it or not the computer is much interesting then in class all day reading out of books. If your child is home on your computer or at a local library, it's better than being out with friends being fresh, or being perpressured to doing something they know isnt right. You might not know where your child is, forbidde in a hospital bed because of a drive-by. Rather than your child on the computer learning, chatting or just playing games, safe and sound in your home or community place. Now I hope you have reached a point to understand and agree with me, because computers can have great effects on you or child because it gives us time to chat with friends/new people, helps us learn about the globe and believe or not keeps us out of troble. Thank you for listening.
1 Dear @CAPS1 @CAPS2, I believe that using computers will benefit us in many ways like talking and becoming friends will others through websites like facebook and mysace. Using computers can help us find coordibates, locations, and able ourselfs to millions of information. Also computers will benefit us by helping with jobs as in planning a house plan and typing a @NUM1 page report for one of our jobs in less than writing it. Now lets go into the wonder world of technology. Using a computer will help us in life by talking or making friends on line. Many people have myspace, facebooks, aim, these all benefit us by having conversations with one another. Many people believe computers are bad but how can you make friends if

you can never talk to them? I am very fortunate for having a computer that can he lp with not only school work but my social life and how I make friends. Computers help us with finding our locations, coordibates and millions of information onlin e. If we didn't go on the internet a lot we wouldn't know how to go onto websites that @MONTH1 help us with locations and coordinates like @LOCATION1. Would you ra ther use a computer or be in @LOCATION3. When your supposed to be vacationing in @LOCATION2. Million of information is found on the internet. You can as almost ev ery question and a computer will have it. Would you rather easily draw up a house plan on the computers or take @NUM1 hours doing one by hand with ugly erazer mark s all over it, you are garrenteed that to find a job with a drawing like that. Al so when appling for a job many workers must write very long papers like a @NUM3 w ord essay on why this job fits you the most, and many people I know don't like wr iting @NUM3 words non-stopp for hours when it could take them I hav an a compute r. That is why computers we needed a lot now adays. I hope this essay has impacte d your descion on computers because they are great machines to work with. The oth er day I showed my mom how to use a computer and she said it was the greatest inv ention sense sliced bread! Now go out and buy a computer to help you chat online with friends, find locations and millions of information on one click of the butt on and help your self with getting a job with neat, prepared, printed work that y our boss will love. Dear , I believe that using computers will benefit us in man y ways like talking and becoming friends will others through websites like facebo ok and mysace. Using computers can help us find coordibates, locations, and able ourselfs to millions of information. Also computers will benefit us by helping wi th jobs as in planning a house plan and typing a page report for one of our jobs in less than writing it. Now lets go into the wonder world of technology. Using a computer will help us in life by talking or making friends on line. Many people h ave myspace, facebooks, aim, these all benefit us by having conversations with on e another. Many people believe computers are bad but how can you make friends if you can never talk to them? I am very fortunate for having a computer that can he lp with not only school work but my social life and how I make friends. Computers help us with finding our locations, coordibates and millions of information onlin e. If we didn't go on the internet a lot we wouldn't know how to go onto websites that help us with locations and coordinates like . Would you rather use a compute r or be in . When your supposed to be vacationing in . Million of information is found on the internet. You can as almost every question and a computer will have it. Would you rather easily draw up a house plan on the computers or take hours d oing one by hand with ugly erazer marks all over it, you are garrenteed that to f ind a job with a drawing like that. Also when appling for a job many workers must write very long papers like a word essay on why this job fits you the most, and m any people I know don't like writing words non-stopp for hours when it could take them I hav an a computer. That is why computers we needed a lot now adays. I hope this essay has impacted your descion on computers because they are great machines to work with. The other day I showed my mom how to use a computer and she said it was the greatest invention sense sliced bread! Now go out and buy a computer to h elp you chat online with friends, find locations and millions of information on o ne click of the button and help your self with getting a job with neat, prepared, printed work that your boss will love.

2

Dear, @CAPS1 @CAPS2 @CAPS3 More and more people use computers, but not everyone a grees that this benefits society. Those who support advances in technology believ e that computers have a positive effect on people. Others have different ideas. A great amount in the world today are using computers, some for work and spme for t he fun of it. Computers is one of mans greatest accomplishments. Computers are he lpful in so many ways, @CAPS4, news, and live streams. Don't get me wrong way to much people spend time on the computer and they should be out interacting with ot hers but who are we to tell them what to do. When I grow up I want to be a author or a journalist and I know for a fact that both of those jobs involve lots of tim e on time on the computer, one @MONTH1 spend more time then the other but you kno w exactly what @CAPS5 getting at. So what if some expert think people are spendin g to much time on the computer and not exercising, enjoying natures and interacti

```
ng with family and friends. For all the expert knows that its how must people mak
e a living and we don't know why people choose to use the computer for a great am
ount of time and to be honest it's non of my concern and it shouldn't be the so c
alled experts concern. People interact a thousand times a day on the computers. C
omputers keep lots of kids of the streets instead of being out and causing troubl
e. Computers helps the @ORGANIZATION1 locate most wanted criminals. As you can se
e computers are more useful to society then you think, computers benefit society.
Dear, More and more people use computers, but not everyone agrees that this benef
its society. Those who support advances in technology believe that computers have
a positive effect on people. Others have different ideas. A great amount in the w
orld today are using computers, some for work and spme for the fun of it. Compute
rs is one of mans greatest accomplishments. Computers are helpful in so many way
s, , news, and live streams. Don't get me wrong way to much people spend time on
the computer and they should be out interacting with others but who are we to tel
l them what to do. When I grow up I want to be a author or a journalist and I kno
w for a fact that both of those jobs involve lots of time on time on the compute
r, one spend more time then the other but you know exactly what getting at. So wh
at if some expert think people are spending to much time on the computer and not
exercising, enjoying natures and interacting with family and friends. For all the
expert knows that its how must people make a living and we don't know why people
choose to use the computer for a great amount of time and to be honest it's non o
f my concern and it shouldn't be the so called experts concern. People interact a
thousand times a day on the computers. Computers keep lots of kids of the streets
instead of being out and causing trouble. Computers helps the locate most wanted
criminals. As you can see computers are more useful to society then you think, co
mputers benefit society.
```

In [10]:
```
print(df[['essay', 'clean_essay']].iloc[0].to_string())
```

```
essay          Dear local newspaper, I think effects computer...
clean_essay    Dear local newspaper, I think effects computer...
```

In [11]:
```
import re

def extract_features(text):
    sentences = re.split(r'[.!?]+', text)
    sentences = [s.strip() for s in sentences if s.strip()]

    words = text.split()

    return {
        'word_count': len(words),
        'sentence_count': len(sentences),
        'avg_sentence_length': len(words) / len(sentences) if sentences else 0,
        'unique_word_count': len(set(word.lower() for word in words)),
        'vocabulary_richness': len(set(word.lower() for word in words)) / len(wo
    }

features = df['clean_essay'].apply(extract_features).apply(pd.Series)

df = pd.concat([df, features], axis=1)

print(df[['essay_set', 'word_count', 'sentence_count',
          'avg_sentence_length', 'unique_word_count',
          'vocabulary_richness']].head())
```

```
   essay_set  word_count  sentence_count  avg_sentence_length  \
0          1       337.0            16.0            21.062500
1          1       413.0            20.0            20.650000
2          1       273.0            14.0            19.500000
3          1       488.0            27.0            18.074074
4          1       462.0            30.0            15.400000

   unique_word_count  vocabulary_richness
0              174.0             0.516320
1              201.0             0.486683
2              152.0             0.556777
3              245.0             0.502049
4              214.0             0.463203
```

In [12]:
```
print(
    df[['word_count',
        'sentence_count',
        'avg_sentence_length',
        'unique_word_count',
        'vocabulary_richness',
        'domain1_score']].corr()['domain1_score'].sort_values(ascending=False)
)
```

```
domain1_score          1.000000
sentence_count         0.610159
unique_word_count      0.583974
word_count             0.576738
avg_sentence_length   -0.101790
vocabulary_richness   -0.396016
Name: domain1_score, dtype: float64
```

In [13]:
```
df['normalized_score'] = df.groupby('essay_set')['domain1_score'].transform(
    lambda x: (x - x.min()) / (x.max() - x.min())
)

print(df[['essay_set', 'domain1_score', 'normalized_score']].head(10))
```

```
   essay_set  domain1_score  normalized_score
0          1              8               0.6
1          1              9               0.7
2          1              7               0.5
3          1             10               0.8
4          1              8               0.6
5          1              8               0.6
6          1             10               0.8
7          1             10               0.8
8          1              9               0.7
9          1              9               0.7
```

In [14]:
```
print(
    df.groupby('essay_set')['normalized_score']
      .agg(['min', 'max', 'mean'])
)
```

```
            min  max      mean
essay_set
1           0.0  1.0  0.652832
2           0.0  1.0  0.483111
3           0.0  1.0  0.616068
4           0.0  1.0  0.477401
5           0.0  1.0  0.602216
6           0.0  1.0  0.680000
7           0.0  1.0  0.639203
8           0.0  1.0  0.539004
```

In [15]:
```
pip install scikit-learn
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: scikit-learn in C:\Users\HP\AppData\Roaming\Python
\Python314\site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in C:\Users\HP\AppData\Roaming\Pytho
n\Python314\site-packages (from scikit-learn) (2.5.2)
Requirement already satisfied: scipy>=1.10.0 in C:\Users\HP\AppData\Roaming\Pytho
n\Python314\site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in C:\Users\HP\AppData\Roaming\Pytho
n\Python314\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\HP\AppData\Roaming\Pyt
hon\Python314\site-packages (from scikit-learn) (2.26.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Users\HP\AppData\Roamin
g\Python\Python314\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in C:\Users\HP\AppData\Roaming\Py
thon\Python314\site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
Note: you may need to restart the kernel to use updated packages.
```

In [16]:
```
from sklearn.model_selection import train_test_split
X = df['clean_essay']
y = df['normalized_score']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("Training essays:", len(X_train))
print("Testing essays:", len(X_test))
```

```
Training essays: 10380
Testing essays: 2596
```

In [17]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(
    max_features=10000,
    stop_words='english',
    ngram_range=(1, 2)
)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("Training shape:", X_train_tfidf.shape)
print("Testing shape:", X_test_tfidf.shape)
```

```
Training shape: (10380, 10000)
Testing shape: (2596, 10000)
```

In [18]:
```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)

model.fit(
    X_train_tfidf,
    y_train
)

y_pred = model.predict(
    X_test_tfidf
)

print("Model trained successfully!")
print("First 10 predictions:")
print(y_pred[:10])
```

```
Model trained successfully!
First 10 predictions:
[0.35465463 0.57114992 0.65441808 0.38063864 0.41119117 0.1606044
 0.42462884 0.86359344 0.79579993 0.6648511 ]
```

In [19]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
correlation = np.corrcoef(y_test, y_pred)[0, 1]
print("MAE:", mae)
print("RMSE:", rmse)
print("Correlation:", correlation)
```

```
MAE: 0.12590777225637576
RMSE: 0.16233453469236564
Correlation: 0.7397564009586368
```

In [20]:
```
!pip install textstat
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: textstat in C:\Users\HP\AppData\Roaming\Python\Pyt
hon314\site-packages (0.7.13)
Requirement already satisfied: pyphen in C:\Users\HP\AppData\Roaming\Python\Pytho
n314\site-packages (from textstat) (0.18.1)
Requirement already satisfied: nltk in C:\Users\HP\AppData\Roaming\Python\Python3
14\site-packages (from textstat) (3.10.3)
Requirement already satisfied: setuptools in C:\Users\HP\AppData\Roaming\Python\P
ython314\site-packages (from textstat) (84.0.0)
Requirement already satisfied: defusedxml in C:\Users\HP\AppData\Roaming\Python\P
ython314\site-packages (from nltk->textstat) (0.7.1)
Requirement already satisfied: click in C:\Users\HP\AppData\Roaming\Python\Python
314\site-packages (from nltk->textstat) (8.5.0)
Requirement already satisfied: joblib in C:\Users\HP\AppData\Roaming\Python\Pytho
n314\site-packages (from nltk->textstat) (1.6.0)
Requirement already satisfied: regex>=2021.8.3 in C:\Users\HP\AppData\Roaming\Pyt
hon\Python314\site-packages (from nltk->textstat) (2026.9.10)
Requirement already satisfied: tqdm in C:\Users\HP\AppData\Roaming\Python\Python3
14\site-packages (from nltk->textstat) (4.70.1)
Requirement already satisfied: cloudpickle>=3.0 in C:\Users\HP\AppData\Roaming\Py
thon\Python314\site-packages (from joblib->nltk->textstat) (3.1.2)
Requirement already satisfied: colorama in C:\Users\HP\AppData\Roaming\Python\Pyt
hon314\site-packages (from tqdm->nltk->textstat) (0.4.6)
```

In [21]:
```
import textstat
```

```
textstat.set_lang("en")

df['readability_score'] = df['clean_essay'].apply(
    lambda x: textstat.flesch_reading_ease(x)
)

print(df[['essay_set', 'word_count', 'readability_score', 'domain1_score']].head
```

```
   essay_set  word_count  readability_score  domain1_score
0          1       337.0          72.264162              8
1          1       413.0          65.900744              9
2          1       273.0          65.191471              7
3          1       488.0          60.866218             10
4          1       462.0          67.916358              8
5          1       239.0          77.572464              8
6          1       491.0          67.199615             10
7          1       476.0          70.397176             10
8          1       439.0          74.521725              9
9          1       495.0          75.223689              9
```

In [22]:
```
correlation = df['readability_score'].corr(df['domain1_score'])

print("Readability vs Essay Score Correlation:", correlation)
```

```
Readability vs Essay Score Correlation: 0.30623084906262826
```

In [23]:
```
!pip install language-tool-python
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: language-tool-python in C:\Users\HP\AppData\Roamin
g\Python\Python314\site-packages (3.4.0)
Requirement already satisfied: requests in C:\Users\HP\AppData\Roaming\Python\Pyt
hon314\site-packages (from language-tool-python) (2.34.2)
Requirement already satisfied: tqdm in C:\Users\HP\AppData\Roaming\Python\Python3
14\site-packages (from language-tool-python) (4.70.1)
Requirement already satisfied: packaging in C:\Users\HP\AppData\Roaming\Python\Py
thon314\site-packages (from language-tool-python) (26.3)
Requirement already satisfied: psutil in C:\Users\HP\AppData\Roaming\Python\Pytho
n314\site-packages (from language-tool-python) (7.2.2)
Requirement already satisfied: toml in C:\Users\HP\AppData\Roaming\Python\Python3
14\site-packages (from language-tool-python) (0.10.2)
Requirement already satisfied: charset_normalizer<4,>=2 in C:\Users\HP\AppData\Ro
aming\Python\Python314\site-packages (from requests->language-tool-python) (3.4.
9)
Requirement already satisfied: idna<4,>=2.5 in C:\Users\HP\AppData\Roaming\Python
\Python314\site-packages (from requests->language-tool-python) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in C:\Users\HP\AppData\Roaming\Py
thon\Python314\site-packages (from requests->language-tool-python) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in C:\Users\HP\AppData\Roaming\P
ython\Python314\site-packages (from requests->language-tool-python) (2026.7.22)
Requirement already satisfied: colorama in C:\Users\HP\AppData\Roaming\Python\Pyt
hon314\site-packages (from tqdm->language-tool-python) (0.4.6)
```

In [24]:
```
import re

def grammar_features(text):
    sentences = re.split(r'[.!?]+', text)
    sentences = [s.strip() for s in sentences if s.strip()]

    words = text.split()
```

```
    # Basic punctuation counts
    comma_count = text.count(',')
    question_count = text.count('?')
    exclamation_count = text.count('!')

    # Very long sentences
    long_sentences = sum(
        len(sentence.split()) > 30
        for sentence in sentences
    )

    # Repeated words
    word_list = [word.lower().strip(".,!?;:\"'()") for word in words]

    repeated_words = sum(
        word_list[i] == word_list[i-1]
        for i in range(1, len(word_list))
        if word_list[i]
    )

    return {
        'comma_count': comma_count,
        'question_count': question_count,
        'exclamation_count': exclamation_count,
        'long_sentence_count': long_sentences,
        'repeated_word_count': repeated_words
    }

grammar_df = df['clean_essay'].apply(grammar_features).apply(pd.Series)

df = pd.concat([df, grammar_df], axis=1)

print(df[
    ['essay_set', 'comma_count', 'question_count',
     'exclamation_count', 'long_sentence_count',
     'repeated_word_count', 'domain1_score']
].head(10))
```

```
   essay_set  comma_count  question_count  exclamation_count  \
0          1           18               2                  4
1          1           12               1                  1
2          1            9               0                  0
3          1           13               1                  2
4          1           13               0                  0
5          1            3               2                  1
6          1           35               4                  0
7          1           11               3                  1
8          1           16               6                  1
9          1           14               2                  0

   long_sentence_count  repeated_word_count  domain1_score
0                    5                    0              8
1                    3                    0              9
2                    3                    1              7
3                    3                    0             10
4                    2                    0              8
5                    1                    0              8
6                    0                    0             10
7                    0                    0             10
8                    1                    0              9
9                    3                    1              9
```

In [25]:
```
feature_columns = [
    'word_count',
    'sentence_count',
    'avg_sentence_length',
    'unique_word_count',
    'vocabulary_richness',
    'readability_score',
    'comma_count',
    'question_count',
    'exclamation_count',
    'long_sentence_count',
    'repeated_word_count'
]
correlations = df[feature_columns + ['domain1_score']].corr()['domain1_score']
correlations = correlations.sort_values(ascending=False)
print(correlations)
```

```
domain1_score          1.000000
sentence_count         0.610159
unique_word_count      0.583974
word_count             0.576738
comma_count            0.412042
exclamation_count      0.320621
readability_score      0.306231
repeated_word_count    0.258212
long_sentence_count    0.242687
question_count         0.175218
avg_sentence_length   -0.101790
vocabulary_richness   -0.396016
Name: domain1_score, dtype: float64
```

In [26]:
```
from scipy.sparse import hstack
from sklearn.preprocessing import StandardScaler
numeric_features = [
    'word_count',
    'sentence_count',
```

```
    'avg_sentence_length',
    'unique_word_count',
    'vocabulary_richness',
    'readability_score',
    'comma_count',
    'question_count',
    'exclamation_count',
    'long_sentence_count',
    'repeated_word_count'
]
scaler = StandardScaler()
X_train_numeric = scaler.fit_transform(
    df.loc[X_train.index, numeric_features]
)
X_test_numeric = scaler.transform(
    df.loc[X_test.index, numeric_features]
)
X_train_combined = hstack([
    X_train_tfidf,
    X_train_numeric
])
X_test_combined = hstack([
    X_test_tfidf,
    X_test_numeric
])
print("Combined training shape:", X_train_combined.shape)
print("Combined testing shape:", X_test_combined.shape)
```

```
Combined training shape: (10380, 10011)
Combined testing shape: (2596, 10011)
```

In [27]:
```
from sklearn.linear_model import Ridge

combined_model = Ridge(alpha=1.0)

combined_model.fit(X_train_combined, y_train)

y_pred_combined = combined_model.predict(X_test_combined)

print("Combined model trained successfully!")
print("First 10 predictions:")
print(y_pred_combined[:10])
```

```
Combined model trained successfully!
First 10 predictions:
[0.37376534 0.58518617 0.61496387 0.36158843 0.43559805 0.15781872
 0.43947817 0.90996335 0.838716   0.71692438]
```

In [28]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
mae_combined = mean_absolute_error(y_test, y_pred_combined)
rmse_combined = np.sqrt(mean_squared_error(y_test, y_pred_combined))
correlation_combined = np.corrcoef(y_test, y_pred_combined)[0, 1]
print("Combined Model Results")
print("MAE:", mae_combined)
print("RMSE:", rmse_combined)
print("Correlation:", correlation_combined)
```

```
Combined Model Results
MAE: 0.11961206434404936
RMSE: 0.15473375152863575
Correlation: 0.7674771446688486
```

In [29]:
```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
rf_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train_numeric, y_train)
y_pred_rf = rf_model.predict(X_test_numeric)
mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
correlation_rf = np.corrcoef(y_test, y_pred_rf)[0, 1]
print("Random Forest Results")
print("MAE:", mae_rf)
print("RMSE:", rmse_rf)
print("Correlation:", correlation_rf)
```

```
Random Forest Results
MAE: 0.13833099723850878
RMSE: 0.17958156436854433
Correlation: 0.6680059639675296
```

In [30]:
```python
def analyze_essay(essay):
    features = extract_features(essay)
    grammar = grammar_features(essay)

    return {
        **features,
        **grammar
    }

sample_essay = df['clean_essay'].iloc[0]

analysis = analyze_essay(sample_essay)

print(analysis)
```

```
{'word_count': 337, 'sentence_count': 16, 'avg_sentence_length': 21.0625, 'unique
_word_count': 174, 'vocabulary_richness': 0.516320474777448, 'comma_count': 18,
'question_count': 2, 'exclamation_count': 4, 'long_sentence_count': 5, 'repeated_
word_count': 0}
```

In [31]:
```python
def generate_rule_feedback(features):
    feedback = []

    # Sentence structure
    if features['long_sentence_count'] > 3:
        feedback.append(
            "Some sentences are quite long. Consider splitting them "
            "into shorter sentences for better clarity."
        )

    # Vocabulary
    if features['vocabulary_richness'] < 0.35:
        feedback.append(
```

```
            "Your vocabulary is somewhat repetitive. Try using a wider "
            "range of words where appropriate."
        )
    else:
        feedback.append(
            "Your essay shows a good variety of vocabulary."
        )

    # Essay length
    if features['word_count'] < 200:
        feedback.append(
            "The essay is relatively short. Consider developing your "
            "ideas with more explanation or examples."
        )
    elif features['word_count'] > 700:
        feedback.append(
            "The essay is quite long. Consider removing unnecessary "
            "repetition and keeping the argument focused."
        )
    else:
        feedback.append(
            "The essay has a reasonable length for developing its ideas."
        )

    # Repeated words
    if features['repeated_word_count'] > 2:
        feedback.append(
            "Some words are repeated consecutively. Review these "
            "sentences for smoother writing."
        )

    # Questions
    if features['question_count'] > 3:
        feedback.append(
            "There are several questions in the essay. Make sure each "
            "one contributes to your main argument."
        )

    # Exclamation marks
    if features['exclamation_count'] > 3:
        feedback.append(
            "There are several exclamation marks. In formal essays, "
            "use them sparingly."
        )

    return feedback


feedback = generate_rule_feedback(analysis)

for i, item in enumerate(feedback, 1):
    print(f"{i}. {item}")
```

```
1. Some sentences are quite long. Consider splitting them into shorter sentences for better clarity.
2. Your essay shows a good variety of vocabulary.
3. The essay has a reasonable length for developing its ideas.
4. There are several exclamation marks. In formal essays, use them sparingly.
```

In [32]:
```
essay = df['clean_essay'].iloc[0]
prompt = f"""
You are an expert English writing evaluator.
Analyze the following student essay.
Evaluate it on these categories:
1. Grammar
2. Sentence Structure
3. Coherence and Flow
4. Argument Strength
5. Vocabulary
6. Overall Writing Quality
For each category:- Give a score from 1 to 10- Give short constructive feedback
Then provide:- Overall score- 3 specific improvement suggestions
Student Essay:
{essay}
"""
print(prompt)
```

```
You are an expert English writing evaluator.
Analyze the following student essay.
Evaluate it on these categories:
1. Grammar
2. Sentence Structure
3. Coherence and Flow
4. Argument Strength
5. Vocabulary
6. Overall Writing Quality
For each category:- Give a score from 1 to 10- Give short constructive feedback
Then provide:- Overall score- 3 specific improvement suggestions
Student Essay:
Dear local newspaper, I think effects computers have on people are great learning
skills/affects because they give us time to chat with friends/new people, helps u
s learn about the globe(astronomy) and keeps us out of troble! Thing about! Dont
you think so? How would you feel if your teenager is always on the phone with fri
ends! Do you ever time to chat with your friends or buisness partner about thing
s. Well now - there's a new way to chat the computer, theirs plenty of sites on t
he internet to do so: , , , facebook, myspace ect. Just think now while your sett
ing up meeting with your boss on the computer, your teenager is having fun on the
phone not rushing to get off cause you want to use it. How did you learn about ot
her countrys/states outside of yours? Well I have by computer/internet, it's a ne
w way to learn about what going on in our time! You might think your child spends
a lot of time on the computer, but ask them so question about the economy, sea fl
oor spreading or even about the 's you'll be surprise at how much he/she knows. B
elieve it or not the computer is much interesting then in class all day reading o
ut of books. If your child is home on your computer or at a local library, it's b
etter than being out with friends being fresh, or being perpressured to doing som
ething they know isnt right. You might not know where your child is, forbidde in
a hospital bed because of a drive-by. Rather than your child on the computer lear
ning, chatting or just playing games, safe and sound in your home or community pl
ace. Now I hope you have reached a point to understand and agree with me, because
computers can have great effects on you or child because it gives us time to chat
with friends/new people, helps us learn about the globe and believe or not keeps
us out of troble. Thank you for listening.
```

In [33]:
```
import sys
print(sys.version)
```

```
3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)]
```

In [34]:
```
from ollama import Client
import os
print("Ollama Python package installed!")
```

```
Ollama Python package installed!
```

In [35]:
```
import ollama
response = ollama.chat(
    model='llama3.2:3b',
    messages=[
        {
            'role': 'user',
            'content': f"""
You are an expert English writing evaluator.
Analyze the following student essay.
Evaluate it on:
1. Grammar
2. Sentence Structure
3. Coherence and Flow
4. Argument Strength
5. Vocabulary
6. Overall Writing Quality
For each category:
 Give a score from 1 to 10- Give short, constructive feedback
Then provide:- Overall score- 3 specific improvement suggestions
Student Essay:
{essay}
"""
}
]
)
print(response['message']['content'])
```

Grammar Score: 4/10

The essay contains numerous grammatical errors, such as subject-verb agreement is sues, incorrect sentence structure, and misspelled words (e.g., "troble" instead of "trouble"). For example, "I think effects computers have on people are great l earning skills/affects" should be rewritten as "I think that effects of computers on people are great learning skills." Correcting these errors will improve the ov erall clarity and professionalism of the essay.

Sentence Structure Score: 3/10

The essay features short, simplistic sentences that lack variety and complexity. Many sentences are also fragmented or contain multiple clauses, making it difficu lt to follow the author's train of thought. To improve sentence structure, the au thor should aim to craft more cohesive and nuanced sentences that convey their id eas more effectively.

Coherence and Flow Score: 5/10

The essay struggles to maintain a clear and logical flow, with abrupt transitions between ideas and lack of cohesion between paragraphs. The author jumps between d ifferent topics, such as the benefits of computers, the importance of education, and the concerns about teenagers' screen time, without clear connections between them. To improve coherence and flow, the author should organize their ideas more logically and use transitional phrases to guide the reader through the essay.

Argument Strength Score: 6/10

The author raises some valid points about the benefits of computers, such as prov iding opportunities for socialization, education, and learning. However, their ar gument is often unclear, and they fail to provide concrete evidence or examples t o support their claims. Additionally, they introduce new ideas (e.g., the importa nce of learning about other countries) without adequately addressing potential co unterarguments. To strengthen their argument, the author should focus on developi ng a clearer, more structured argument with supporting evidence and addressing po tential criticisms.

Vocabulary Score: 2/10

The author's vocabulary is limited, and they often use simplistic or incorrect wo rds (e.g., "fresh" instead of "freshen up"). The writing lacks variety, with repe titive use of basic words and phrases (e.g., "because it gives us time to chat wi th friends"). To improve vocabulary, the author should strive to use more precis e, nuanced words that convey their ideas more effectively.

Overall Writing Quality Score: 4/10

The essay demonstrates a lack of polish and refinement, with numerous errors in g rammar, sentence structure, and vocabulary. While the author raises some valid po ints, their writing style is often unclear, simplistic, and lacking in depth. To improve overall writing quality, the author should focus on refining their writin g skills, using more precise language, and organizing their ideas more effectivel y.

Three Specific Improvement Suggestions:

1. **Edit for grammatical errors**: Carefully review the essay for grammatical er rors, subject-verb agreement issues, and other mistakes. Make revisions to ensure that the writing is clear, concise, and free of errors.
2. **Develop a clearer argument structure**: Organize the essay around a clear ar

gument or thesis statement. Use transitional phrases to guide the reader through the essay and ensure that the ideas flow logically. Focus on developing a more st ructured argument with supporting evidence and addressing potential counterargume nts.
3. **Enhance vocabulary and writing style**: Strive to use more precise, nuanced words that convey the author's ideas more effectively. Vary sentence structure an d vocabulary to create a more engaging, polished writing style. Avoid repetitive use of basic words and phrases, and focus on using more sophisticated language to express complex ideas.

In [36]:
```
import ollama

response = ollama.chat(
    model='llama3.2:3b',
    messages=[
        {
            'role': 'user',
            'content': f"""
You are an expert English writing evaluator.

Analyze this student essay and return ONLY valid JSON.

Use exactly this structure:

{{
  "grammar": {{
    "score": 0,
    "feedback": ""
  }},
  "sentence_structure": {{
    "score": 0,
    "feedback": ""
  }},
  "coherence": {{
    "score": 0,
    "feedback": ""
  }},
  "argument_strength": {{
    "score": 0,
    "feedback": ""
  }},
  "vocabulary": {{
    "score": 0,
    "feedback": ""
  }},
  "overall": {{
    "score": 0,
    "feedback": ""
  }},
  "suggestions": [
    "",
    "",
    ""
  ]
}}

Rules:
- Scores must be between 1 and 10.
- Feedback must be concise and constructive.
- Suggestions must be specific and actionable.
```

```
- Do not include Markdown.
- Do not include anything outside the JSON.

Student Essay:
{essay}
"""
        }
    ]
)

print(response['message']['content'])
```


```
{
  "grammar": {
    "score": 2,
    "feedback": "The essay contains several grammatical errors, including subject
-verb agreement issues, incorrect use of verb tenses, and punctuation mistakes. E
nsure proper grammar and punctuation throughout the essay."
  },
  "sentence_structure": {
    "score": 4,
    "feedback": "The essay lacks varied sentence structures, with many sentences
being similar in length and structure. Varying sentence structure can improve the
essay's flow and readability."
  },
  "coherence": {
    "score": 6,
    "feedback": "The essay's ideas are somewhat coherent, but they are not fully
developed or supported with evidence. Consider adding more specific examples and
supporting evidence to strengthen the argument."
  },
  "argument_strength": {
    "score": 3,
    "feedback": "The essay presents a clear argument, but it lacks depth and nuan
ce. Consider adding more complexity and counterarguments to strengthen the argume
nt."
  },
  "vocabulary": {
    "score": 5,
    "feedback": "The essay uses some advanced vocabulary, but it could benefit fr
om more precise and nuanced language. Consider using more descriptive words to en
hance the writing."
  },
  "overall": {
    "score": 5,
    "feedback": "The essay demonstrates a clear argument, but it could benefit fr
om improvement in grammar, sentence structure, coherence, and argument strength.
Consider revising the essay to address these areas."
  },
  "suggestions": [
    "Use specific examples and evidence to support the argument.",
    "Revise sentence structures to vary length and structure.",
    "Add more complexity and nuance to the argument by considering counterargumen
ts."
  ]
}
```


In [37]:
```
import json

feedback_text = response['message']['content']
```

```
feedback = json.loads(feedback_text)

print("JSON parsed successfully!")
print(feedback)
```

```
JSON parsed successfully!
{'grammar': {'score': 2, 'feedback': 'The essay contains several grammatical erro
rs, including subject-verb agreement issues, incorrect use of verb tenses, and pu
nctuation mistakes. Ensure proper grammar and punctuation throughout the essa
y.'}, 'sentence_structure': {'score': 4, 'feedback': "The essay lacks varied sent
ence structures, with many sentences being similar in length and structure. Varyi
ng sentence structure can improve the essay's flow and readability."}, 'coherenc
e': {'score': 6, 'feedback': "The essay's ideas are somewhat coherent, but they a
re not fully developed or supported with evidence. Consider adding more specific
examples and supporting evidence to strengthen the argument."}, 'argument_strengt
h': {'score': 3, 'feedback': 'The essay presents a clear argument, but it lacks d
epth and nuance. Consider adding more complexity and counterarguments to strength
en the argument.'}, 'vocabulary': {'score': 5, 'feedback': 'The essay uses some a
dvanced vocabulary, but it could benefit from more precise and nuanced language.
Consider using more descriptive words to enhance the writing.'}, 'overall': {'sco
re': 5, 'feedback': 'The essay demonstrates a clear argument, but it could benefi
t from improvement in grammar, sentence structure, coherence, and argument streng
th. Consider revising the essay to address these areas.'}, 'suggestions': ['Use s
pecific examples and evidence to support the argument.', 'Revise sentence structu
res to vary length and structure.', 'Add more complexity and nuance to the argume
nt by considering counterarguments.']}
```

In [38]:
```
print("Grammar Score:", feedback["grammar"]["score"])
print("Coherence Score:", feedback["coherence"]["score"])
print("Overall Score:", feedback["overall"]["score"])

print("\nSuggestions:")
for suggestion in feedback["suggestions"]:
    print("-", suggestion)
```

```
Grammar Score: 2
Coherence Score: 6
Overall Score: 5

Suggestions:
- Use specific examples and evidence to support the argument.
- Revise sentence structures to vary length and structure.
- Add more complexity and nuance to the argument by considering counterarguments.
```

In [39]:
```
def evaluate_essay(essay_text, essay_set):
    # 1. Clean essay
    clean_text = clean_essay(essay_text)
    # 2. Extract NLP features
    features = analyze_essay(clean_text)
    # 3. Create numeric features
    numeric_values = pd.DataFrame([[
        features['word_count'],
        features['sentence_count'],
        features['avg_sentence_length'],
        features['unique_word_count'],
        features['vocabulary_richness'],
        textstat.flesch_reading_ease(clean_text),
        features['comma_count'],
        features['question_count'],
        features['exclamation_count'],
```

```
        features['long_sentence_count'],
        features['repeated_word_count']
    ]], columns=numeric_features)
    # 4. Scale numeric features
    numeric_values = scaler.transform(numeric_values)
    # 5. TF-IDF features
    essay_tfidf = vectorizer.transform([clean_text])
    # 6. Combine TF-IDF + NLP features
    essay_combined = hstack([
        essay_tfidf,
        numeric_values
    ])
    # 7. Select the correct essay-set model
    if essay_set not in essay_set_models:
        raise ValueError(
            f"Invalid essay set. Choose one of: "
            f"{list(essay_set_models.keys())}"
        )
    selected_model = essay_set_models[essay_set]
    # 8. Predict ML score
    predicted_score = selected_model.predict(
        essay_combined
    )[0]
    # Keep prediction between 0 and 1
    predicted_score = max(0, min(1, predicted_score))
    predicted_percentage = predicted_score * 100
    # 9. Generate AI feedback
    ai_response = ollama.chat(
        model='llama3.2:3b',
        messages=[
            {
                'role': 'user',
                'content': f"""
You are an expert English writing evaluator.
Analyze the student essay and return ONLY valid JSON.
Use exactly this structure:
{{
  "grammar": {{
    "score": 0,
    "feedback": ""
  }},
  "sentence_structure": {{
    "score": 0,
     "feedback": ""
  }},
  "coherence": {{
    "score": 0,
    "feedback": ""
  }},
  "argument_strength": {{
    "score": 0,
    "feedback": ""
  }},
  "vocabulary": {{
    "score": 0,
    "feedback": ""
  }},
  "overall": {{
    "score": 0,
    "feedback": ""
```

```
  }},
  "suggestions": [
    "",
    "",
    ""
  ]
}}
Rules:- Scores must be between 1 and 10.- Give concise and constructive feedback
Student Essay:
{clean_text}
"""
            }
        ]
    )
    # 10. Convert AI response to dictionary
    feedback_text = ai_response['message']['content']
    feedback = json.loads(feedback_text)
    # 11. Add ML prediction
    feedback["ml_predicted_score"] = round(
        predicted_percentage,
        2
    )
    return feedback
```

In [58]:

```
result = evaluate_essay(new_essay, 1)
print(json.dumps(result, indent=2))
```

```
{
  "grammar": {
    "score": 9,
    "feedback": ""
  },
  "sentence_structure": {
    "score": 9,
    "feedback": ""
  },
  "coherence": {
    "score": 9,
    "feedback": ""
  },
  "argument_strength": {
    "score": 8,
    "feedback": "The essay presents a clear and well-supported argument about the role of technology in modern education. However, some paragraphs could be more explicitly linked to the thesis statement."
  },
  "vocabulary": {
    "score": 8,
    "feedback": "The vocabulary is generally strong, but some phrases such as 'makes learning more accessible' could be replaced with more precise language."
  },
  "overall": {
    "score": 8,
    "feedback": "The essay is well-organized and clearly written. However, the introduction could be more engaging and the conclusion could be more detailed."
  },
  "suggestions": [
    "Consider adding more specific examples or anecdotes to support the argument in the introduction.",
    "In the conclusion, consider elaborating on the potential long-term consequences of excessive technology use on students.",
    "In the argument_strength section, consider providing more specific evidence or research to support the claim that technology should be used as a tool to support education, rather than replace traditional learning."
  ],
  "ml_predicted_score": 68.43
}
```


In [41]:

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
gb_model = GradientBoostingRegressor(
n_estimators=200,
learning_rate=0.05,
max_depth=3,
random_state=42
)
gb_model.fit(X_train_numeric, y_train)
y_pred_gb = gb_model.predict(X_test_numeric)
mae_gb = mean_absolute_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
correlation_gb = np.corrcoef(y_test, y_pred_gb)[0, 1]
print("Gradient Boosting Results")
print("--------------------------")
print("MAE:", mae_gb)
print("RMSE:", rmse_gb)
print("Correlation:", correlation_gb)
```

```
Gradient Boosting Results
-------------------------
MAE: 0.1364962072685447
RMSE: 0.17696815931336735
Correlation: 0.679413684714769
```

In [59]:
```python
new_essay = """
Technology has changed the way students learn.
Students can now access educational resources from anywhere.
However, excessive use of technology can reduce concentration.
Therefore, technology should be used carefully in education.
"""
result = evaluate_essay(new_essay,1)
print(json.dumps(result, indent=2))
```


```
{
  "grammar": {
    "score": 9,
    "feedback": ""
  },
  "sentence_structure": {
    "score": 8,
    "feedback": "Consider varying sentence structure to improve flow and engageme
nt."
  },
  "coherence": {
    "score": 9,
    "feedback": ""
  },
  "argument_strength": {
    "score": 7,
    "feedback": "Strengthen your argument by providing evidence or examples to su
pport the claim that technology should be used carefully in education."
  },
  "vocabulary": {
    "score": 8,
    "feedback": "Use more precise vocabulary to convey your ideas, such as \"over
reliance on technology\" instead of \"excessive use of technology\"."
  },
  "overall": {
    "score": 8,
    "feedback": ""
  },
  "suggestions": [
    "Consider adding a counterargument to strengthen your argument.",
    "Use transitional phrases to connect your ideas and improve coherence.",
    "Provide specific examples to illustrate the benefits and drawbacks of using
technology in education."
  ],
  "ml_predicted_score": 27.72
}
```


In [62]:
```python
print("Correlation with word count:")
print(df['word_count'].corr(df['domain1_score']))
print("\nCorrelation with sentence count:")
print(df['sentence_count'].corr(df['domain1_score']))
print("\nCorrelation with vocabulary richness:")
print(df['vocabulary_richness'].corr(df['domain1_score']))
```

```
Correlation with word count:
0.5767380497318282

Correlation with sentence count:
0.6101594526141364

Correlation with vocabulary richness:
-0.396016399770289
```

In [63]:

```python
def advanced_features(text):
    sentences = re.split(r'[.!?]+', text)
    sentences = [s.strip() for s in sentences if s.strip()]

    words = text.split()
    word_list = [
        w.lower().strip(".,!?;:\"'()")
        for w in words
    ]
    word_list = [w for w in word_list if w]
    # Sentence lengths
    sentence_lengths = [len(s.split()) for s in sentences]
    # Transition words
    transition_words = [
        'however', 'therefore', 'moreover', 'furthermore',
        'although', 'because', 'thus', 'consequently',
        'additionally', 'firstly', 'secondly', 'finally',
        'for example', 'for instance', 'in conclusion'
    ]
    transition_count = sum(
        text.lower().count(word)
        for word in transition_words
    )
    # Paragraph count
    paragraph_count = len([
        p for p in text.split('\n')
        if p.strip()
    ])
    # Average word length
    avg_word_length = (
        sum(len(w) for w in word_list) / len(word_list)
        if word_list else 0
    )
    # Sentence length variation
    sentence_length_std = (
        np.std(sentence_lengths)
        if sentence_lengths else 0
    )
    # Punctuation density
    punctuation_count = sum(
        text.count(p)
        for p in [',', '.', ';', ':', '!', '?']
    )
    punctuation_density = (
        punctuation_count / len(words)
        if words else 0
    )
    # Transition density
    transition_density = (
        transition_count / len(words)
        if words else 0
```

```
    )
    return {
        'avg_word_length': avg_word_length,
        'sentence_length_std': sentence_length_std,
        'paragraph_count': paragraph_count,
        'transition_count': transition_count,
        'transition_density': transition_density,
        'punctuation_density': punctuation_density
    }
```

In [64]:
```
advanced_df = (
    df['clean_essay']
    .apply(advanced_features)
    .apply(pd.Series)
)
df = pd.concat([df, advanced_df], axis=1)
print(df[[
    'avg_word_length',
    'sentence_length_std',
    'paragraph_count',
    'transition_count',
    'transition_density',
    'punctuation_density'
]].head())
```

```
   avg_word_length  sentence_length_std  paragraph_count  transition_count  \
0         4.356287            13.131397              1.0               4.0
1         4.325183            11.608187              1.0               1.0
2         4.371324            13.404424              1.0               0.0
3         4.712810            10.477227              1.0               5.0
4         4.405640             6.616142              1.0               4.0

   transition_density  punctuation_density
0            0.011869             0.103858
1            0.002421             0.077482
2            0.000000             0.084249
3            0.010246             0.081967
4            0.008658             0.093074
```

In [65]:
```
print(
df[[
'avg_word_length',
'sentence_length_std',
'paragraph_count',
'transition_count',
'transition_density',
'punctuation_density',
'domain1_score'
]].corr()['domain1_score'].sort_values(ascending=False)
)
```

```
domain1_score          1.000000
transition_count       0.181170
sentence_length_std    0.097295
punctuation_density    0.055809
transition_density    -0.151836
avg_word_length       -0.372469
paragraph_count             NaN
Name: domain1_score, dtype: float64
```

In [45]:
```
from scipy.sparse import hstack
from sklearn.preprocessing import OneHotEncoder
# Encode essay set
encoder = OneHotEncoder(
    sparse_output=True,
    handle_unknown='ignore'
)
train_set = encoder.fit_transform(
    df.loc[X_train.index, ['essay_set']]
)
test_set = encoder.transform(
    df.loc[X_test.index, ['essay_set']]
)
print("Essay set features:", train_set.shape)
```

```
Essay set features: (10380, 8)
```

In [46]:
```
X_train_with_set = hstack([
    X_train_tfidf,
    X_train_numeric,
    train_set
])
X_test_with_set = hstack([
    X_test_tfidf,
    X_test_numeric,
    test_set
])
print("Training shape:", X_train_with_set.shape)
print("Testing shape:", X_test_with_set.shape)
```

```
Training shape: (10380, 10019)
Testing shape: (2596, 10019)
```

In [47]:
```
from sklearn.linear_model import Ridge
ridge_set_model = Ridge(alpha=1.0)
ridge_set_model.fit(
    X_train_with_set,
    y_train
)
y_pred_set = ridge_set_model.predict(
    X_test_with_set
)
```

In [48]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
mae_set = mean_absolute_error(y_test, y_pred_set)
rmse_set = np.sqrt(
    mean_squared_error(y_test, y_pred_set)
)
correlation_set = np.corrcoef(
    y_test,
    y_pred_set
)[0, 1]
print("Ridge + TF-IDF + NLP + Essay Set")
print("--------------------------------")
print("MAE:", mae_set)
print("RMSE:", rmse_set)
print("Correlation:", correlation_set)
```

```
Ridge + TF-IDF + NLP + Essay Set
--------------------------------
MAE: 0.11124077092090466
RMSE: 0.14611441017265792
Correlation: 0.7962311459386585
```

In [49]:
```python
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
alphas = [0.01, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0]
results = []
for alpha in alphas:
    model = Ridge(alpha=alpha)
    model.fit(
        X_train_with_set,
        y_train
    )
    predictions = model.predict(
        X_test_with_set
    )
    mae = mean_absolute_error(y_test, predictions)
    rmse = np.sqrt(
        mean_squared_error(y_test, predictions)
    )
    correlation = np.corrcoef(
        y_test,
        predictions
    )[0, 1]
    results.append({
        "alpha": alpha,
        "MAE": mae,
        "RMSE": rmse,
        "Correlation": correlation
    })
results_df = pd.DataFrame(results)
print(results_df)
```

```
   alpha       MAE      RMSE  Correlation
0   0.01  0.195944  0.247701     0.571902
1   0.10  0.134187  0.172966     0.725760
2   0.50  0.115026  0.150876     0.783055
3   1.00  0.111241  0.146114     0.796231
4   2.00  0.109416  0.143501     0.803691
5   5.00  0.109355  0.142833     0.805935
6  10.00  0.110703  0.144205     0.802372
7  20.00  0.112926  0.147008     0.794641
```

In [50]:
```python
from sklearn.linear_model import Ridge
combined_model = Ridge(alpha=5.0)
combined_model.fit(
    X_train_with_set,
    y_train
)
print("Final Ridge model trained successfully!")
```

```
Final Ridge model trained successfully!
```

In [51]:
```python
print(combined_model.n_features_in_)
```

```
10019
```

In [ ]:
```
new_essay = """
Technology has become an essential part of modern education. It has changed the

One major advantage of technology is that it makes learning more accessible. Stu

However, technology also has some disadvantages when it is used excessively. Spe

Therefore, technology should be used as a tool to support education rather than

In conclusion, technology has the potential to greatly improve education when it
"""
result = evaluate_essay(new_essay, 1)

print(json.dumps(result, indent=2))
```

In [53]:
```
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd

set_results = []

for essay_set in sorted(df['essay_set'].unique()):

    # Get train and test rows for this essay set
    train_idx = X_train.index[
        df.loc[X_train.index, 'essay_set'] == essay_set
    ]

    test_idx = X_test.index[
        df.loc[X_test.index, 'essay_set'] == essay_set
    ]

    # TF-IDF
    X_set_train = vectorizer.transform(
        df.loc[train_idx, 'clean_essay']
    )

    X_set_test = vectorizer.transform(
        df.loc[test_idx, 'clean_essay']
    )

    # Numeric features
    X_num_train = scaler.transform(
        df.loc[train_idx, numeric_features]
    )

    X_num_test = scaler.transform(
        df.loc[test_idx, numeric_features]
    )

    # Combine
    X_set_train_combined = hstack([
        X_set_train,
        X_num_train
    ])

    X_set_test_combined = hstack([
        X_set_test,
```

```
        X_num_test
    ])

    # Train model
    model = Ridge(alpha=5.0)

    model.fit(
        X_set_train_combined,
        y_train.loc[train_idx]
    )

    # Predict
    predictions = model.predict(
        X_set_test_combined
    )

    # Metrics
    mae = mean_absolute_error(
        y_test.loc[test_idx],
        predictions
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_test.loc[test_idx],
            predictions
        )
    )

    correlation = np.corrcoef(
        y_test.loc[test_idx],
        predictions
    )[0, 1]

    set_results.append({
        'Essay Set': essay_set,
        'MAE': mae,
        'RMSE': rmse,
        'Correlation': correlation
    })

set_results_df = pd.DataFrame(set_results)

print(set_results_df)
```

```
   Essay Set       MAE      RMSE  Correlation
0          1  0.059730  0.076015     0.857656
1          2  0.078374  0.100371     0.781669
2          3  0.151264  0.194534     0.701144
3          4  0.131128  0.173535     0.831756
4          5  0.104042  0.129718     0.834317
5          6  0.103685  0.130504     0.850639
6          7  0.103306  0.127960     0.790358
7          8  0.062310  0.079121     0.758508
```

In [54]:

```
all_predictions = []
all_actual = []

for essay_set in sorted(df['essay_set'].unique()):
```

```
    train_idx = X_train.index[
        df.loc[X_train.index, 'essay_set'] == essay_set
    ]

    test_idx = X_test.index[
        df.loc[X_test.index, 'essay_set'] == essay_set
    ]

    X_set_train = vectorizer.transform(
        df.loc[train_idx, 'clean_essay']
    )

    X_set_test = vectorizer.transform(
        df.loc[test_idx, 'clean_essay']
    )

    X_num_train = scaler.transform(
        df.loc[train_idx, numeric_features]
    )

    X_num_test = scaler.transform(
        df.loc[test_idx, numeric_features]
    )

    X_set_train_combined = hstack([
        X_set_train,
        X_num_train
    ])

    X_set_test_combined = hstack([
        X_set_test,
        X_num_test
    ])

    model = Ridge(alpha=5.0)

    model.fit(
        X_set_train_combined,
        y_train.loc[train_idx]
    )

    predictions = model.predict(
        X_set_test_combined
    )

    all_predictions.extend(predictions)
    all_actual.extend(y_test.loc[test_idx])

overall_mae = mean_absolute_error(
    all_actual,
    all_predictions
)

overall_rmse = np.sqrt(
    mean_squared_error(
        all_actual,
        all_predictions
    )
)
```

```
overall_correlation = np.corrcoef(
    all_actual,
    all_predictions
)[0, 1]

print("Separate Essay-Set Models")
print("-------------------------")
print("MAE:", overall_mae)
print("RMSE:", overall_rmse)
print("Correlation:", overall_correlation)
```

```
Separate Essay-Set Models
-------------------------
MAE: 0.10158852072826247
RMSE: 0.13513856071721006
Correlation: 0.8283581945053503
```

In [55]:
```
from sklearn.linear_model import Ridge
essay_set_models = {}
for essay_set in sorted(df['essay_set'].unique()):
    # Get train and test indices
    train_idx = X_train.index[
        df.loc[X_train.index, 'essay_set'] == essay_set
    ]
    # TF-IDF features
    X_set_train = vectorizer.transform(
        df.loc[train_idx, 'clean_essay']
    )
    # Numeric features
    X_num_train = scaler.transform(
        df.loc[train_idx, numeric_features]
    )
    # Combine features
    X_set_train_combined = hstack([
        X_set_train,
        X_num_train
    ])
    # Train model
    model = Ridge(alpha=5.0)
    model.fit(
        X_set_train_combined,
        y_train.loc[train_idx]
    )
    # Store model
    essay_set_models[essay_set] = model
print("Models trained successfully!")
print("Number of models:", len(essay_set_models))
print("Available essay sets:", list(essay_set_models.keys()))
```

```
Models trained successfully!
Number of models: 8
Available essay sets: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int
64(5), np.int64(6), np.int64(7), np.int64(8)]
```

In [56]:
```
import joblib
joblib.dump(vectorizer, "tfidf_vectorizer.pkl")
joblib.dump(scaler, "feature_scaler.pkl")
joblib.dump(essay_set_models, "essay_set_models.pkl")
print("All ML components saved successfully!")
```

```
All ML components saved successfully!
```

In [57]:
```
import joblib
all_models = {
    "Ridge - TF-IDF": model,
    "Ridge - TF-IDF + NLP Features": combined_model,
    "Random Forest": rf_model,
    "Gradient Boosting": gb_model,
    "Ridge - Separate Essay Set": essay_set_models
}
joblib.dump(all_models, "all_models.pkl")
print("All models saved successfully!")
```

```
All models saved successfully!
```

In [61]:
```
import joblib

from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from scipy.sparse import hstack


# ============================================================
# 1. RIDGE - TF-IDF ONLY
# Expected input features = 10,000
# ============================================================

ridge_tfidf_model = Ridge(alpha=1.0)

ridge_tfidf_model.fit(
    X_train_tfidf,
    y_train
)

print("Ridge TF-IDF trained")


# ============================================================
# 2. RIDGE - TF-IDF + NLP FEATURES
# Expected input features = 10,011
# ============================================================

ridge_nlp_model = Ridge(alpha=1.0)

ridge_nlp_model.fit(
    X_train_combined,
    y_train
)

print("Ridge TF-IDF + NLP trained")


# ============================================================
# 3. RANDOM FOREST
# Expected input features = 11
# ============================================================

rf_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
```

```
rf_model.fit(
    X_train_numeric,
    y_train
)

print("Random Forest trained")


# ============================================================
# 4. GRADIENT BOOSTING
# Expected input features = 11
# ============================================================

gb_model = GradientBoostingRegressor(
    n_estimators=100,
    random_state=42
)

gb_model.fit(
    X_train_numeric,
    y_train
)

print("Gradient Boosting trained")


# ============================================================
# 5. SEPARATE RIDGE MODEL FOR EACH ESSAY SET
# Expected input features = 10,011
# ============================================================

essay_set_models = {}

for essay_set in sorted(df['essay_set'].unique()):

    # Get training rows belonging to this essay set
    train_idx = X_train.index[
        df.loc[X_train.index, 'essay_set'] == essay_set
    ]

    # TF-IDF
    X_set_train_tfidf = vectorizer.transform(
        df.loc[train_idx, 'clean_essay']
    )

    # Numeric features
    X_set_train_numeric = scaler.transform(
        df.loc[train_idx, numeric_features]
    )

    # Combine TF-IDF + NLP
    X_set_train_combined = hstack([
        X_set_train_tfidf,
        X_set_train_numeric
    ])

    # Separate Ridge model
    set_model = Ridge(alpha=5.0)
```

```
    set_model.fit(
        X_set_train_combined,
        y_train.loc[train_idx]
    )

    essay_set_models[essay_set] = set_model

print("Separate Essay Set models trained")
print("Available essay sets:", list(essay_set_models.keys()))


# ============================================================
# 6. SAVE INDIVIDUAL COMPONENTS
# ============================================================

joblib.dump(
    vectorizer,
    "tfidf_vectorizer.pkl"
)

joblib.dump(
    scaler,
    "feature_scaler.pkl"
)

joblib.dump(
    essay_set_models,
    "essay_set_models.pkl"
)


# ============================================================
# 7. SAVE ALL MODELS CORRECTLY
# ============================================================

all_models = {

    "Ridge - TF-IDF":
        ridge_tfidf_model,

    "Ridge - TF-IDF + NLP Features":
        ridge_nlp_model,

    "Random Forest":
        rf_model,

    "Gradient Boosting":
        gb_model,

    "Ridge - Separate Essay Set":
        essay_set_models
}


joblib.dump(
    all_models,
    "all_models.pkl"
)
```

```
print("\n====================================")
print("ALL MODELS SAVED SUCCESSFULLY!")
print("====================================")

print("\nModel feature counts:")

print(
    "Ridge - TF-IDF:",
    ridge_tfidf_model.n_features_in_
)

print(
    "Ridge - TF-IDF + NLP Features:",
    ridge_nlp_model.n_features_in_
)

print(
    "Random Forest:",
    rf_model.n_features_in_
)

print(
    "Gradient Boosting:",
    gb_model.n_features_in_
)

print(
    "Separate Ridge models: 10,011 features each"
)

print("\nFiles created:")
print("tfidf_vectorizer.pkl")
print("feature_scaler.pkl")
print("essay_set_models.pkl")
print("all_models.pkl")
```

```
Ridge TF-IDF trained
Ridge TF-IDF + NLP trained
Random Forest trained
Gradient Boosting trained
Separate Essay Set models trained
Available essay sets: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int
64(5), np.int64(6), np.int64(7), np.int64(8)]

====================================
ALL MODELS SAVED SUCCESSFULLY!
====================================

Model feature counts:
Ridge - TF-IDF: 10000
Ridge - TF-IDF + NLP Features: 10011
Random Forest: 11
Gradient Boosting: 11
Separate Ridge models: 10,011 features each

Files created:
tfidf_vectorizer.pkl
feature_scaler.pkl
essay_set_models.pkl
all_models.pkl
```